

Strings & String Builder

- * String represents text as an immutable sequence of characters.
- * StringBuilder is a mutable alternative used when text needs to change frequently.

When to use:

- * Use String for text that won't change often: names, messages, fixed labels.
- * Use StringBuilder when you need to build or modify text repeatedly - especially inside loops (Ex: constructing a large report, concatenating many pieces).

How to use:

A) String Basics.

```
String name = "Rahul";  
name.length();  
name.charAt(0);  
name.toUpperCase();  
name.toLowerCase();  
name.substring(1, 3);  
name.indexOf("h");  
name.equals("Rahul");  
name.equalsIgnoreCase("rahul");  
name.concat("Sharma");  
name.replace("R", "M");  
name.trim();  
name.split(" ");
```

B. String Builder

```
StringBuilder sb = new  
    StringBuilder("Hello")  
sb.append("world");  
sb.insert(5, " ,");  
sb.reverse();  
sb.delete(0, 5);  
sb.replace(0, 1, "J");  
String result = sb.toString();
```

Where to use:

- * String - almost everywhere text is used.
- * StringBuilder - Inside loops that build text piece by piece (ex: generating a CSV row, constructing HTML, repeated concatenation).

Why:

- * String is immutable - "modifying" it creates a new object each time; old one is discarded; Enables Safe Sharing (String pool, thread safe, security).
- * Concatenating Strings in a loop (result += i) is inefficient - creates a new object every iteration (O(n²)).
- * StringBuilder uses an internal resizable buffer, modified in place - much faster for repeated changes (O(n)).
- * Use .equals() (Content comparison), not == (reference/memory comparison), to compare String values.

Real-life Example:

<u>Scenario</u>	<u>Tool</u>
Fixed name/email	String
Comparing passwords	.equals()
Building chat log/report line by line	StringBuilder
Reversing a word	StringBuilder.reverse()